

Universidad Católica de la Santísima Concepción

Facultad de Ingeniería
Ingeniería Civil Informática

Informe de Laboratorio

Tarea 2: Aplicación Cliente-Servidor mediante Sockets TCP

Asignatura: Redes de Computadores (2026)

Profesor: Yasmany García Prieto

Integrantes: Oscar Gonzalez
Jeremy Mendoza
Franco Videla

Fecha: 24 de junio de 2026

Índice

1. Descripción de la Aplicación	2
2. Motivación y Utilidad	2
3. Herramientas Utilizadas	2
4. Descripción del Código	3
4.1. Servidor (<code>servidor_C.c</code>)	3
4.2. Cliente (<code>Cliente_C.c</code>)	4
4.3. Estructuras de Datos Principales	5
5. Justificación del Uso de TCP	5
6. Capturas de Pantalla de Operación	6
7. Capturas de Paquetes en Wireshark	6
7.1. Establecimiento de la Conexión — Three-Way Handshake	7
7.2. Comando <code>listar</code> — Solicitud del Cliente	8
7.3. Respuesta al Comando <code>listar</code>	9
7.4. Comando <code>descargar</code> — Solicitud del Cliente	9
7.5. Señal de Fin de Transferencia	10
7.6. Comando <code>terminar()</code> ; — Cierre de Sesión	10
8. Alcance y Posibles Mejoras	11
8.1. Alcance Actual	11
8.2. Limitaciones Identificadas	11
8.3. Posibles Mejoras	11
9. Reflexión sobre la Experiencia	12
10. Referencias	12

1. Descripción de la Aplicación

La aplicación desarrollada es un **sistema de transferencia de archivos cliente-servidor** implementado en lenguaje C, basado en la API de sockets de Berkeley para sistemas Unix/Linux. El sistema permite a un cliente conectarse a un servidor remoto a través de una red TCP/IP e interactuar con él mediante un conjunto de comandos predefinidos.

El servidor escucha conexiones entrantes en un puerto configurable y atiende a los clientes de forma secuencial. Una vez establecida la conexión, el cliente puede ejecutar los siguientes comandos:

- **listar** — solicita al servidor el listado de todos los archivos presentes en su directorio de trabajo.
- **descargar <nombre_archivo>** — solicita la transferencia de un archivo específico desde el servidor hacia el cliente, donde se guarda localmente con el prefijo **copia_**.
- **terminar();** — indica al servidor que la sesión ha finalizado y cierra la conexión de forma ordenada.

2. Motivación y Utilidad

La motivación principal detrás del desarrollo de esta aplicación es comprender de manera práctica el funcionamiento del modelo cliente-servidor y el rol que cumplen los sockets como interfaz de programación de red. Aplicaciones de este tipo constituyen la base de herramientas ampliamente utilizadas como FTP (*File Transfer Protocol*), SCP y NFS.

La utilidad concreta de la aplicación reside en:

- Permitir el acceso remoto a archivos almacenados en un servidor, con posibilidad de listarlos y descargarlos.
- Servir como base para sistemas de distribución de contenido o servidores de repositorios simples.
- Demostrar la viabilidad de implementar transferencia de datos confiable mediante sockets TCP en redes heterogéneas.

3. Herramientas Utilizadas

Lenguaje C (estándar POSIX)

Implementación del cliente y servidor utilizando la API de sockets Berkeley (`sys/socket.h`, `netinet/in.h`, `arpa/inet.h`).

GCC (GNU Compiler Collection)

Compilador utilizado para generar los ejecutables del cliente y el servidor bajo Linux.

Ubuntu Linux

Sistema operativo empleado tanto en la máquina del servidor como en la del cliente durante las pruebas.

VirtualBox

Herramienta de virtualización utilizada para simular máquinas independientes en la misma red física, permitiendo probar la comunicación entre cliente y servidor en entornos separados.

Wireshark

Analizador de protocolos de red utilizado para capturar e inspeccionar los paquetes TCP intercambiados durante la comunicación entre cliente y servidor.

Terminal / Bash

Entorno de ejecución para compilar, desplegar y probar los programas.

4. Descripción del Código

4.1. Servidor (servidor_C.c)

El servidor sigue el flujo estándar de un servidor TCP pasivo:

1. **Creación del socket** con `socket(AF_INET, SOCK_STREAM, 0)`, indicando familia IPv4 y tipo de flujo orientado a conexión.
2. **Configuración de la dirección** (`sockaddr_in`) con el puerto recibido como parámetro y la dirección `INADDR_ANY` para escuchar en todas las interfaces.
3. **Enlace** mediante `bind()` entre el socket y la dirección local.
4. **Modo escucha** con `listen()`, configurando una cola de hasta 100 conexiones pendientes.
5. **Aceptación de conexiones** con `accept()`, bloqueándose hasta que un cliente se conecte.
6. **Procesamiento de comandos** en un bucle `do-while` hasta recibir `terminar();`.

La lógica de comandos se implementa con comparaciones de cadena:

```
1  if(strncmp(mensaje_entrada, "listar", 6) == 0) {
2      // Abre el directorio actual y construye la lista de
        archivos
3      DIR *d = opendir(".");
4      while((dir = readdir(d)) != NULL) {
```

```
5         strcat(mensaje_salida, dir->d_name);
6         strcat(mensaje_salida, "\n");
7     }
8 } else if(strncmp(mensaje_entrada, "descargar", 9) == 0) {
9     // Extrae nombre de archivo y lo transfiere por bloques
10    sscanf(mensaje_entrada, "descargar %s", nombre_archivo);
11    FILE *fp = fopen(nombre_archivo, "r");
12    while((bytes_leidos = fread(msg, 1, MAX_TAM_MENSAJE, fp))
13          > 0)
14        send(descriptor_socket_cliente, msg, bytes_leidos, 0);
15    send(descriptor_socket_cliente, "Fin", 4, 0); // Señal de
16                                                fin
17 } else {
18     strcpy(mensaje_salida, "Comando no reconocido\n");
19 }
```

Listing 1: Procesamiento de comandos en el servidor

La señal SIGINT (Ctrl+C) es capturada por la función `catch()`, la cual cierra ambos descriptors de socket antes de terminar el proceso, garantizando una liberación ordenada de recursos.

4.2. Cliente (Cliente_C.c)

El cliente implementa el rol activo de la conexión TCP:

1. **Creación del socket** y enlace a un puerto efímero mediante `bind()` con puerto 0.
2. **Configuración del destino** con la IP y puerto recibidos como argumentos (`argv[1]`, `argv[2]`).
3. **Conexión** al servidor con `connect()`.
4. **Bucle de interacción**: el usuario ingresa un comando con `fgets()`, se envía al servidor con `send()`, y se recibe la respuesta con `recv()`.

Para el comando `descargar`, el cliente implementa una recepción por bloques:

```
1 FILE *fp = fopen(nombre_local, "w");
2 while ((bytes = recv(descriptor_socket_origen,
3                     respuesta, MAX_TAM_MENSAJE, 0)) > 0) {
4     if(strstr(respuesta, "Fin") != NULL) break;
5     fwrite(respuesta, 1, bytes, fp);
6 }
7 fclose(fp);
```

Listing 2: Recepción de archivo en el cliente

La señal `SIGINT` en el cliente escribe `terminar()`; en el buffer del mensaje, forzando el cierre ordenado de la sesión.

4.3. Estructuras de Datos Principales

`struct sockaddr_in`

Estructura estándar POSIX que almacena la familia de direcciones (`sin_family`), la dirección IP (`sin_addr`) y el puerto (`sin_port`) de un extremo de la conexión.

`int descriptor_socket`

Entero que actúa como identificador del socket. Los descriptors del servidor y del cliente activo se almacenan como variables globales para poder cerrarlos desde el manejador de señal.

`char mensaje[MAX_TAM_MENSAJE]`

Buffer de 512 bytes usado para enviar y recibir datos. Esta limitación implica que archivos mayores a 512 bytes se transfieren en múltiples segmentos TCP.

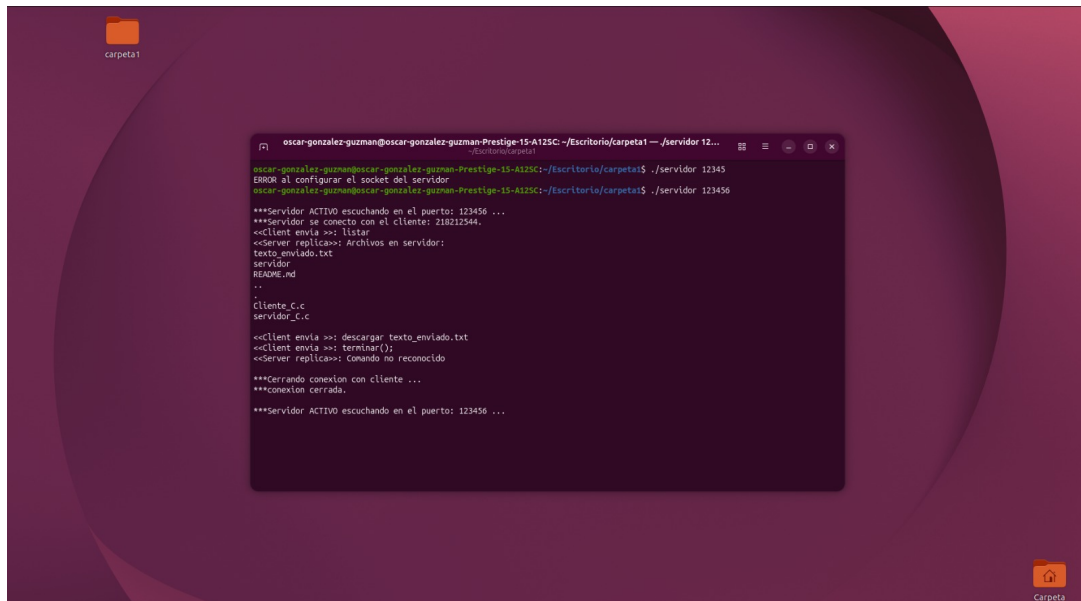
5. Justificación del Uso de TCP

Se utilizó el protocolo **TCP** (**T**ransmission **C**ontrol **P**rotocol) en lugar de UDP por las siguientes razones:

1. **Confiabilidad en la transferencia de archivos:** TCP garantiza que todos los segmentos enviados lleguen al destino, en orden y sin duplicados, mediante el mecanismo de reconocimientos (*ACK*) y retransmisión. Esto es crítico para la integridad de los archivos transferidos con el comando **descargar**.
2. **Control de flujo y congestión:** TCP ajusta automáticamente la tasa de transmisión en función de la capacidad del receptor y del estado de la red, evitando la pérdida de datos por desbordamiento de buffers.
3. **Orientado a conexión:** El establecimiento previo de la conexión mediante el *Three-Way Handshake* garantiza que ambos extremos estén disponibles antes de iniciar el intercambio de datos.
4. **Inapropiado para UDP:** UDP sería adecuado para aplicaciones que priorizan la latencia sobre la integridad (audio/video en tiempo real, DNS). En un sistema de transferencia de archivos, la pérdida de un solo datagrama corrompería el archivo descargado, haciendo a UDP inadecuado sin implementar un protocolo de confiabilidad propio por encima de él.

La elección de `SOCK_STREAM` en la llamada a `socket()` refleja directamente esta decisión de diseño.

6. Capturas de Pantalla de Operación



```
oscar-gonzalez-guzman@oscar-gonzalez-guzman-Prestige-15-A125C: ~/Escritorio/carpetas1 -- /servidor 12...
oscar-gonzalez-guzman@oscar-gonzalez-guzman-Prestige-15-A125C: ~/Escritorio/carpetas1$ ./servidor 12345
ERROR al configurar el socket del servidor
oscar-gonzalez-guzman@oscar-gonzalez-guzman-Prestige-15-A125C: ~/Escritorio/carpetas1$ ./servidor 123456

***Servidor ACTIVO escuchando en el puerto: 123456 ...
***Servidor se conecto con el cliente: 218212544.
<<Client envia >>> listar
<<Server replica>>> Archivos en servidor:
texto_enviado.txt
servidor
README.md
..
.
Cliente_C.c
servidor_C.c

<<Client envia >>> descargar texto_enviado.txt
<<Client envia >>> terminar();
<<Server replica>>> Comando no reconocido
***Cerrando conexión con cliente ...
***conexión cerrada.
***Servidor ACTIVO escuchando en el puerto: 123456 ...
```

Figura 1: Terminal con comandos que interactúa cliente-servidor. Se aprecia el primer intento fallido con el puerto 12345 (ya en uso), el inicio exitoso en el puerto 123456, la conexión con el cliente, la recepción del comando `listar`, la solicitud de descarga de `texto_enviado.txt` y el envío de `terminar()`; con el cierre ordenado de la conexión.

7. Capturas de Paquetes en Wireshark

Todas las capturas corresponden al flujo TCP filtrado con `tcp.stream eq 3` entre las direcciones **192.168.1.12** (cliente, puerto 57920) y **192.168.1.13** (servidor, puerto 50945).

7.1. Establecimiento de la Conexión — Three-Way Handshake

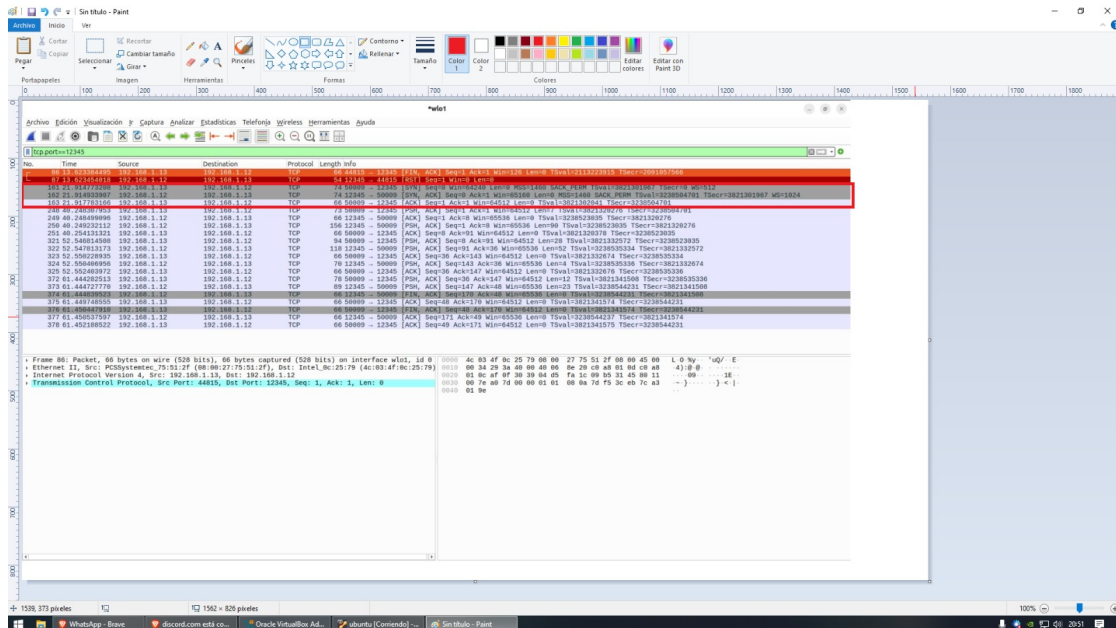


Figura 2: Establecimiento de la conexión TCP entre cliente y servidor mediante el proceso *Three-Way Handshake*, garantizando una comunicación confiable antes del intercambio de datos.

Los tres primeros paquetes corresponden al proceso de establecimiento:

- **SYN** (paquete 112): el cliente solicita la conexión enviando un segmento con el flag SYN activado e indicando su número de secuencia inicial.
- **SYN-ACK** (paquete 113): el servidor acepta la conexión respondiendo con SYN+ACK, confirmando el número de secuencia del cliente e indicando el suyo.
- **ACK** (paquete 125): el cliente confirma la recepción del SYN-ACK, completando el handshake y habilitando el flujo de datos bidireccional.

7.2. Comando listar — Solicitud del Cliente

No.	Time	Source	Destination	Protocol	Length	Info
114	38.622945876	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2458062485 TSecr=3383865991
215	57.153295815	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=36 Ack=143 Win=64512 Len=0 TSval=2458089014 TSecr=3383832519
217	57.157217378	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=36 Ack=147 Win=64512 Len=0 TSval=2458089018 TSecr=3383832524
231	62.169665749	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=48 Ack=170 Win=64512 Len=0 TSval=2458094032 TSecr=3383837537
127	34.107666532	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=8 Ack=91 Win=64512 Len=0 TSval=2458065967 TSecr=3383899473
232	62.169671444	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [FIN, ACK] Seq=8 Ack=171 Win=64512 Len=0 TSval=2458094032 TSecr=3383837537
124	34.101486595	192.168.1.13	192.168.1.12	TCP	73	56945 → 57920 [PSH, ACK] Seq=1 Ack=1 Win=64512 Len=7 TSval=2458065897 TSecr=3383865991
225	62.166278262	192.168.1.13	192.168.1.12	TCP	78	56945 → 57920 [PSH, ACK] Seq=36 Ack=147 Win=64512 Len=12 TSval=2458093977 TSecr=3383832524
213	57.148438801	192.168.1.13	192.168.1.12	TCP	84	56945 → 57920 [PSH, ACK] Seq=8 Ack=91 Win=64512 Len=28 TSval=2458089095 TSecr=3383899473
112	38.620981170	192.168.1.13	192.168.1.12	TCP	74	56945 → 57920 [SYN] Seq=0 Win=64248 Len=0 MSS=1460 SACK_PERM TSval=2458062450 TSecr=0 WS=512
123	34.101618974	192.168.1.12	192.168.1.13	TCP	66	57920 → 56945 [ACK] Seq=1 Ack=8 Win=65536 Len=0 TSval=3383899472 TSecr=2458065897
233	62.169787739	192.168.1.12	192.168.1.13	TCP	66	57920 → 56945 [ACK] Seq=171 Ack=49 Win=65536 Len=0 TSval=3383837548 TSecr=2458094032
238	62.166721546	192.168.1.12	192.168.1.13	TCP	66	57920 → 56945 [FIN, ACK] Seq=170 Ack=49 Win=65536 Len=0 TSval=3383837537 TSecr=2458093977
120	34.102356061	192.168.1.12	192.168.1.13	TCP	150	57920 → 56945 [PSH, ACK] Seq=1 Ack=8 Win=65536 Len=90 TSval=3383899473 TSecr=2458065897
216	57.153336593	192.168.1.12	192.168.1.13	TCP	70	57920 → 56945 [PSH, ACK] Seq=143 Ack=36 Win=65536 Len=4 TSval=3383832524 TSecr=2458089014
229	62.166592540	192.168.1.12	192.168.1.13	TCP	89	57920 → 56945 [PSH, ACK] Seq=147 Ack=48 Win=65536 Len=23 TSval=3383837537 TSecr=2458093977
214	57.149064662	192.168.1.12	192.168.1.13	TCP	118	57920 → 56945 [PSH, ACK] Seq=91 Ack=36 Win=65536 Len=52 TSval=3383832519 TSecr=2458089095
113	38.620284008	192.168.1.12	192.168.1.13	TCP	74	57920 → 56945 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=3383865991 TSecr=2458062450 WS=1024

Frame 124: Packet, 73 bytes on wire (584 bits), 73 bytes captured (584 bits) on interface wlo1, id 6	0000	4c 03 4f 0c 25 79 08 00 27 75 51 2f 08 00 45 00	L 0 %y... 'uQ/ E
» Ethernet II, Src: PCSByteLine_75:51:2f (08:00:27:75:51:2f), Dst: Intel_0c:25:79 (4c:03:4f:0c:25:79)	0010	00 3b 00 39 40 00 40 06 31 1a c0 a0 01 00 c0 a0	; 90 0 1 ...
» Internet Protocol Version 4, Src: 192.168.1.13, Dst: 192.168.1.12	0020	01 0c c7 01 e2 49 08 15 da e3 ae 7b 13 23 86 18	... @ : ... { # :
» Transmission Control Protocol, Src Port: 56945, Dst Port: 57920, Seq: 1, Ack: 1, Len: 7	0030	00 7e dc 1b 00 09 01 01 08 0a 92 83 1b e9 c4 ec	
» Data (7 bytes)	0040	14 27 0c 69 73 74 61 72 00	..lstar..

Figura 3: Envío del comando `lstar` desde el cliente hacia el servidor mediante un segmento TCP. El contenido del paquete puede visualizarse en formato ASCII, donde se aprecia la cadena `lstar`, utilizada para solicitar al servidor el listado de archivos disponibles.

El flag **PSH** indica que el receptor debe entregar inmediatamente los datos al proceso de aplicación sin retenerlos en buffer.

7.3. Respuesta al Comando listar

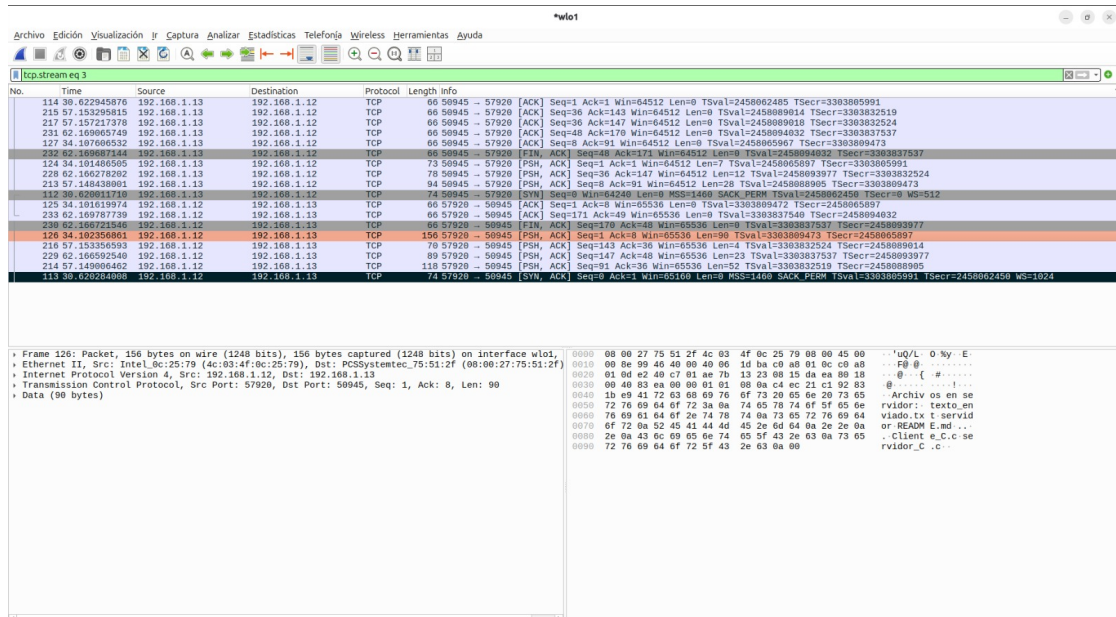


Figura 4: Respuesta del servidor al comando `listar`. El servidor envía al cliente el listado de archivos disponibles en el directorio compartido utilizando la conexión TCP establecida previamente. En la sección de datos del paquete se observa el contenido transmitido en formato ASCII.

7.4. Comando descargar — Solicitud del Cliente

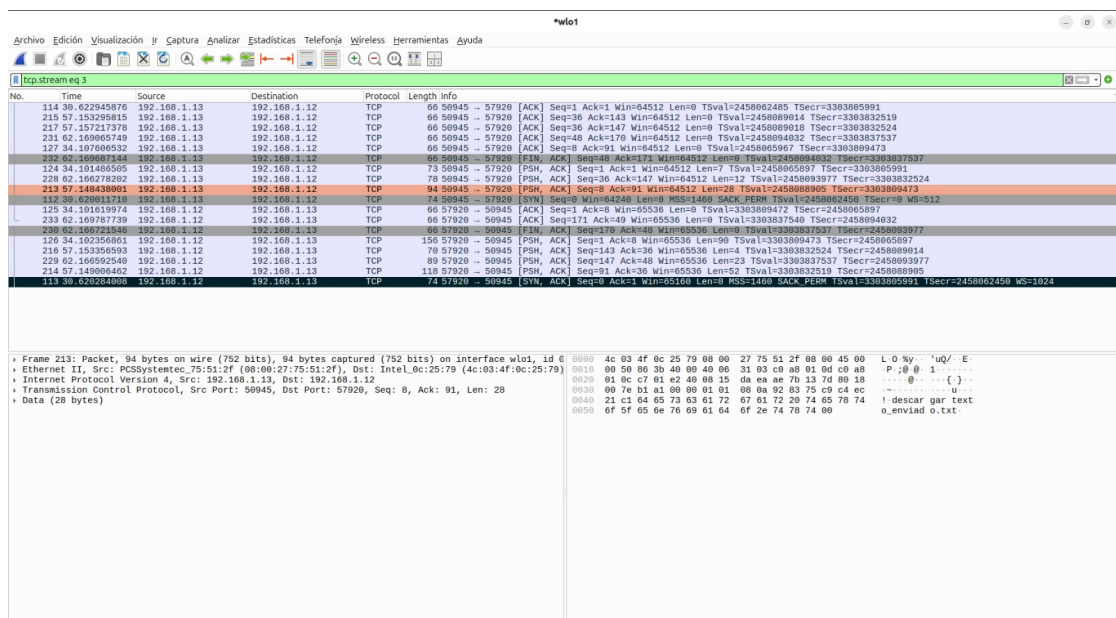


Figura 5: Solicitud de descarga enviada desde el cliente al servidor. El comando `descargar texto_enviado.txt` es transportado dentro de un segmento TCP, permitiendo al servidor identificar el archivo solicitado para iniciar su transferencia.

7.5. Señal de Fin de Transferencia

No.	Time	Source	Destination	Protocol	Length	Info
114	30.622945876	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2458062485 TSecr=3303805991
213	57.153225815	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=36 Ack=143 Win=64512 Len=0 TSval=2458089014 TSecr=3303832519
217	57.157217378	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=36 Ack=147 Win=64512 Len=0 TSval=2458089018 TSecr=3303832524
231	62.169605749	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=48 Ack=170 Win=64512 Len=0 TSval=2458094032 TSecr=3303837537
127	34.107606532	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=8 Ack=91 Win=64512 Len=0 TSval=2458065967 TSecr=3303809473
232	62.169607444	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [FIN, ACK] Seq=48 Ack=171 Win=64512 Len=0 TSval=2458094032 TSecr=3303837537
124	34.101406505	192.168.1.13	192.168.1.12	TCP	73	56945 → 57920 [PSH, ACK] Seq=1 Ack=1 Win=64512 Len=7 TSval=2458065897 TSecr=3303805991
228	62.166278202	192.168.1.13	192.168.1.12	TCP	78	56945 → 57920 [PSH, ACK] Seq=36 Ack=147 Win=64512 Len=12 TSval=2458093977 TSecr=3303832524
213	57.148438001	192.168.1.13	192.168.1.12	TCP	84	56945 → 57920 [PSH, ACK] Seq=8 Ack=91 Win=64512 Len=28 TSval=2458088905 TSecr=3303809473
125	34.101619974	192.168.1.12	192.168.1.13	TCP	66	57920 → 56945 [ACK] Seq=171 Ack=49 Win=65536 Len=0 TSval=3303837540 TSecr=2458094032
233	62.169787739	192.168.1.12	192.168.1.13	TCP	66	57920 → 56945 [ACK] Seq=171 Ack=49 Win=65536 Len=0 TSval=3303837540 TSecr=2458094032
126	34.102356001	192.168.1.12	192.168.1.13	TCP	150	57920 → 56945 [PSH, ACK] Seq=1 Ack=8 Win=65536 Len=90 TSval=3303809473 TSecr=2458065897
218	57.153336593	192.168.1.12	192.168.1.13	TCP	89	57920 → 56945 [PSH, ACK] Seq=143 Ack=36 Win=65536 Len=4 TSval=3303837537 TSecr=2458093977
228	62.166052540	192.168.1.12	192.168.1.13	TCP	89	57920 → 56945 [PSH, ACK] Seq=143 Ack=36 Win=65536 Len=4 TSval=3303837537 TSecr=2458093977
214	57.149006402	192.168.1.12	192.168.1.13	TCP	118	57920 → 56945 [PSH, ACK] Seq=91 Ack=36 Win=65536 Len=52 TSval=3303832519 TSecr=2458088905
113	39.620264008	192.168.1.12	192.168.1.13	TCP	74	57920 → 56945 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=3303805991 TSecr=2458062450 WS=1024

Frame 216: Packet, 78 bytes on wire (560 bits), 78 bytes captured (560 bits) on interface wlo1, id 60000, 08 00 27 75 51 2f 4c 03 4f 0c 25 79 08 00 45 00 → u/L 0 % E

Ethernet II, Src: Intel_0c:25:79 (4c:03:4f:0c:25:79), Dst: PCSysntec_75:51:2f (08:00:27:75:51:2f)

Internet Protocol Version 4, Src: 192.168.1.12, Dst: 192.168.1.13

Transmission Control Protocol, Src Port: 57920, Dst Port: 56945, Seq: 143, Ack: 36, Len: 4

Data (4 bytes)

0000 08 00 27 75 51 2f 4c 03 4f 0c 25 79 08 00 45 00 → u/L 0 % E

0010 00 38 99 48 40 00 40 00 1e 0e c0 a8 01 0c c0 a8 → 0 H0 0
 0020 01 0d e2 49 c7 01 ae 7b 13 d1 08 15 db 06 08 18 → 0 0 0 0 0 0 0 0
 0030 00 a8 63 94 00 00 01 01 08 0a c4 ec 7b cc 92 83 → 0 0 0 0 0 0 0 0
 0040 7b 36 46 09 0e 00 → vGFin

Figura 6: Finalización de la transferencia del archivo. El servidor envía la cadena **Fin** para indicar al cliente que todos los datos del archivo solicitado han sido transmitidos correctamente.

7.6. Comando `terminar()`; — Cierre de Sesión

No.	Time	Source	Destination	Protocol	Length	Info
114	30.622945876	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2458062485 TSecr=3303805991
213	57.153225815	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=36 Ack=143 Win=64512 Len=0 TSval=2458089014 TSecr=3303832519
217	57.157217378	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=36 Ack=147 Win=64512 Len=0 TSval=2458089018 TSecr=3303832524
231	62.169605749	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=48 Ack=170 Win=64512 Len=0 TSval=2458094032 TSecr=3303837537
127	34.107606532	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [ACK] Seq=8 Ack=91 Win=64512 Len=0 TSval=2458065967 TSecr=3303809473
232	62.169607444	192.168.1.13	192.168.1.12	TCP	66	56945 → 57920 [FIN, ACK] Seq=48 Ack=171 Win=64512 Len=0 TSval=2458094032 TSecr=3303837537
124	34.101406505	192.168.1.13	192.168.1.12	TCP	73	56945 → 57920 [PSH, ACK] Seq=1 Ack=1 Win=64512 Len=7 TSval=2458065897 TSecr=3303805991
228	62.166278202	192.168.1.13	192.168.1.12	TCP	84	56945 → 57920 [PSH, ACK] Seq=36 Ack=147 Win=64512 Len=12 TSval=2458093977 TSecr=3303832524
213	57.148438001	192.168.1.13	192.168.1.12	TCP	84	56945 → 57920 [PSH, ACK] Seq=8 Ack=91 Win=64512 Len=28 TSval=2458088905 TSecr=3303809473
125	34.101619974	192.168.1.12	192.168.1.13	TCP	66	57920 → 56945 [ACK] Seq=171 Ack=49 Win=65536 Len=0 TSval=3303837540 TSecr=2458094032
233	62.169787739	192.168.1.12	192.168.1.13	TCP	66	57920 → 56945 [ACK] Seq=171 Ack=49 Win=65536 Len=0 TSval=3303837540 TSecr=2458094032
126	34.102356001	192.168.1.12	192.168.1.13	TCP	150	57920 → 56945 [PSH, ACK] Seq=1 Ack=8 Win=65536 Len=90 TSval=3303809473 TSecr=2458065897
218	57.153336593	192.168.1.12	192.168.1.13	TCP	89	57920 → 56945 [PSH, ACK] Seq=143 Ack=36 Win=65536 Len=4 TSval=3303832524 TSecr=2458093977
228	62.166052540	192.168.1.12	192.168.1.13	TCP	89	57920 → 56945 [PSH, ACK] Seq=143 Ack=36 Win=65536 Len=4 TSval=3303837537 TSecr=2458093977
214	57.149006402	192.168.1.12	192.168.1.13	TCP	118	57920 → 56945 [PSH, ACK] Seq=91 Ack=36 Win=65536 Len=52 TSval=3303832519 TSecr=2458088905
113	39.620264008	192.168.1.12	192.168.1.13	TCP	74	57920 → 56945 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=3303805991 TSecr=2458062450 WS=1024

Frame 228: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface wlo1, id 60000, 4c 03 4f 0c 25 79 08 00 27 75 51 2f 4c 03 4f 0c 25 79 08 00 45 00 → L 0 % 'uQ/ E

Ethernet II, Src: PCSysntec_75:51:2f (08:00:27:75:51:2f), Dst: Intel_0c:25:79 (4c:03:4f:0c:25:79)

Internet Protocol Version 4, Src: 192.168.1.13, Dst: 192.168.1.12

Transmission Control Protocol, Src Port: 56945, Dst Port: 57920, Seq: 36, Ack: 147, Len: 12

Data (12 bytes)

0000 4c 03 4f 0c 25 79 08 00 27 75 51 2f 4c 03 4f 0c 25 79 08 00 45 00 → L 0 % 'uQ/ E

0010 00 a8 06 3e 40 00 40 00 31 19 c9 a8 01 0d c0 a8 → 0 0 0 0 1
 0020 01 0c c7 01 e2 49 08 15 db 06 ae 7b 13 05 08 18 → 0 0 0 0 0 0 0 0
 0030 00 7e 32 7f 00 00 01 01 08 0a 92 83 89 09 4c ec → 0 0 0 0 0 0 0 0
 0040 7b cc 74 65 72 6d 69 0e 61 72 28 28 30 00 → { termin ar();

Figura 7: El cliente envía el comando **terminar()**; para finalizar la comunicación con el servidor. Este mensaje es utilizado por la aplicación para cerrar la sesión y liberar los recursos asociados a la conexión.

Tras este intercambio, el servidor cierra el socket del cliente con `close()` y vuelve al estado de escucha para nuevas conexiones.

8. Alcance y Posibles Mejoras

8.1. Alcance Actual

La aplicación en su estado actual permite:

- Comunicación cliente-servidor en red local o a través de Internet (con el puerto adecuadamente enrutado/abierto en el firewall).
- Listado de archivos del directorio de trabajo del servidor.
- Descarga de archivos de texto plano desde el servidor al cliente.
- Manejo de señales de interrupción (SIGINT) para cierre ordenado.
- Detección básica de errores en las llamadas al sistema de red.

8.2. Limitaciones Identificadas

- **Concurrencia:** el servidor atiende a un cliente a la vez de forma bloqueante.
- **Tamaño de buffer:** el buffer de 512 bytes implica que respuestas muy largas podrían truncarse.
- **Detección de fin de transferencia:** la cadena `Fin` podría aparecer en el contenido legítimo de un archivo binario.
- **Sin autenticación:** cualquier cliente que conozca la IP y el puerto puede conectarse.
- **Solo archivos de texto:** `fopen(..., r)` en modo texto puede alterar archivos binarios.

8.3. Posibles Mejoras

1. **Concurrencia con `fork()` o `pthread`:** atender múltiples clientes simultáneamente.
2. **Protocolo de fin robusto:** usar un campo de longitud en lugar de cadena centinela.
3. **Soporte binario:** abrir archivos en modo `rb/wb`.
4. **Comando subir:** transferencia en sentido cliente $\rightarrow$ servidor.
5. **Autenticación básica:** requerir usuario y contraseña.
6. **Cifrado con TLS:** integrar OpenSSL para proteger los datos en tránsito.

9. Reflexión sobre la Experiencia

El desarrollo de esta tarea permitió consolidar el entendimiento del modelo de comunicación TCP/IP desde la perspectiva del programador de aplicaciones. Entre los aprendizajes más relevantes destacan:

- La diferencia práctica entre un socket activo (cliente) y un socket pasivo (servidor), y el rol de `listen()`/`accept()` en el proceso de establecimiento de conexión.
- La necesidad de diseñar un protocolo a nivel de aplicación para delimitar mensajes y señales de control por sobre el flujo de bytes que provee TCP.
- El uso de Wireshark como herramienta de diagnóstico que permite correlacionar el comportamiento del código con los paquetes reales intercambiados en la red.
- Una dificultad encontrada fue que el puerto configurado inicialmente ya estaba en uso, resultando en un error de `bind()`; esto evidenció la importancia de manejar adecuadamente los errores de todas las llamadas al sistema.

10. Referencias

- **Repositorio GitHub del código fuente:** contiene la implementación en C del cliente y servidor utilizados en la tarea.
<https://github.com/fdjdjhtdjutryr/Tarea-2-Redes-de-computadores>